

Cheatsheet de Arduino

Proyecto: Transformando el Eje Z — Técnica en Sistemas

1. Estructura básica de un programa

Todo programa (sketch) de Arduino tiene dos funciones obligatorias:

```
void setup() {  
  // Se ejecuta UNA sola vez, al encender o resetear la placa.  
  // Aquí van: pinMode(), Serial.begin(), configuraciones iniciales.  
}  
  
void loop() {  
  // Se ejecuta en bucle infinito, una y otra vez.  
  // Aquí va la lógica principal del programa.  
}
```

- Cada línea termina en punto y coma ;
- Los bloques de código van entre llaves { }
- // inicia un comentario de una línea, /* ... */ un comentario de varias líneas

2. Tipos de datos comunes

Tipo	Descripción	Rango aproximado	Ejemplo
int	Número entero	-32,768 a 32,767	int velocidad = 100;
unsigned int	Entero sin signo	0 a 65,535	unsigned int pasos = 500;
long	Entero grande	-2,147,483,648 a 2,147,483,647	long tiempo = 100000;
unsigned long	Entero grande sin signo	0 a 4,294,967,295	unsigned long ahora;
float	Decimal	±3.4e±38 (6-7 dígitos)	float voltaje = 4.8;
double	Decimal (igual a float en Uno)	igual que float	double angulo;
bool / boolean	Verdadero o falso	true / false	bool encendido = true;
char	Un carácter	-128 a 127	char letra = 'A';
byte	Entero pequeño sin signo	0 a 255	byte pin = 9;
String	Cadena de texto (objeto)	—	String msg = "hola";

Tip: String (con mayúscula) es un objeto y consume más memoria. Para proyectos con RAM limitada, es mejor usar arrays de char.

3. Constantes y pines especiales

```
HIGH      // Nivel alto (5V o 3.3V)  
LOW       // Nivel bajo (0V)  
INPUT     // Pin configurado como entrada  
OUTPUT    // Pin configurado como salida
```

```
INPUT_PULLUP // Entrada con resistencia interna a 5V activada
true / false
LED_BUILTIN  // LED integrado en la placa (normalmente pin 13)
```

4. Entradas y salidas digitales

```
pinMode(pin, MODE);          // INPUT, OUTPUT o INPUT_PULLUP
digitalWrite(pin, HIGH);     // Escribe HIGH o LOW en un pin de salida
int valor = digitalRead(pin); // Lee HIGH (1) o LOW (0) de un pin de
                               entrada
```

Ejemplo — encender un LED

```
void setup() {
    pinMode(8, OUTPUT);
}

void loop() {
    digitalWrite(8, HIGH);
    delay(1000);
    digitalWrite(8, LOW);
    delay(1000);
}
```

5. Entradas y salidas analógicas (PWM)

```
int valor = analogRead(pin); // Lee un valor entre 0 y 1023 (pines A0-A5)
analogWrite(pin, valor);     // Escribe PWM entre 0 y 255 (pines con ~)
```

`analogRead()` convierte un voltaje (0-5V) en un número (0-1023): útil para potenciómetros y sensores.
`analogWrite()` no da un voltaje real, simula uno mediante PWM: útil para controlar brillo o velocidad de motores.

Ejemplo — controlar velocidad con potenciómetro

```
void setup() {
    pinMode(9, OUTPUT);
}

void loop() {
    int lectura = analogRead(A0);          // 0 a 1023
    int pwm = map(lectura, 0, 1023, 0, 255); // Reescala a 0-255
    analogWrite(9, pwm);
}
```

6. Comunicación serial (monitor serie)

```
Serial.begin(9600);          // Inicia comunicación (siempre en setup)
Serial.print("Texto");       // Imprime sin salto de línea
Serial.println("Texto");     // Imprime con salto de línea
Serial.print(variable);      // Imprime el valor de una variable
bool hayDatos = Serial.available(); // >0 si hay datos entrantes
char dato = Serial.read();   // Lee un byte recibido
```

Ejemplo — depurar valores en el monitor serie

```
void setup() {
  Serial.begin(9600);
}

void loop() {
  int valor = analogRead(A0);
  Serial.print("Sensor: ");
  Serial.println(valor);
  delay(200);
}
```

7. Control de tiempo

```
delay(ms);           // Pausa el programa X milisegundos (BLOQUEANTE)
delayMicroseconds(us); // Pausa en microsegundos
unsigned long t = millis(); // Milisegundos desde el arranque (no bloquea)
unsigned long t2 = micros(); // Microsegundos desde el arranque
```

Importante: delay() congela todo el programa. Para hacer varias cosas "al mismo tiempo" (ej. leer un sensor mientras un motor gira) se usa millis():

```
unsigned long anterior = 0;
const long intervalo = 1000;

void loop() {
  unsigned long actual = millis();
  if (actual - anterior >= intervalo) {
    anterior = actual;
    // código que se repite cada segundo, SIN bloquear el resto del loop
  }
}
```

8. Estructuras de control

```
// Condicionales
if (condicion) {
  // ...
} else if (otraCondicion) {
  // ...
} else {
  // ...
}

// Switch-case
switch (variable) {
  case 1:
    // ...
    break;
  case 2:
    // ...
    break;
  default:
    // ...
}
```

```

}

// Bucle for
for (int i = 0; i < 10; i++) {
    // ...
}

// Bucle while
while (condicion) {
    // ...
}

// Bucle do-while
do {
    // ...
} while (condicion);

```

9. Operadores

Categoría	Operadores
Aritméticos	+ - * / %
Comparación	== != < > <= >=
Lógicos	&& (y) (o) ! (no)
Asignación compuesta	+= -= *= /=
Incremento / decremento	++ --
Bit a bit	& ^ ~ << >>

10. Funciones matemáticas útiles

```

map(valor, in_min, in_max, out_min, out_max); // Reescala un rango a otro
constrain(valor, min, max); // Limita un valor a un rango
min(a, b); // El menor de dos valores
max(a, b); // El mayor de dos valores
abs(x); // Valor absoluto
pow(base, exponente); // Potencia
sqrt(x); // Raíz cuadrada
random(min, max); // Número aleatorio entre min y max
randomSeed(analogRead(A5)); // "Semilla" para que random() no repita

```

11. Arrays (arreglos)

```

int pines[5] = {2, 3, 4, 5, 6}; // Declarar e inicializar
int primero = pines[0]; // Acceder (empieza en 0)
pines[2] = 9; // Modificar
int tamano = sizeof(pines) / sizeof(pines[0]); // Cantidad de elementos

for (int i = 0; i < tamano; i++) {
    digitalWrite(pines[i], HIGH);
}

```

12. Funciones propias

```

// Función que devuelve un valor

```

```
int sumar(int a, int b) {
    return a + b;
}

// Función que no devuelve nada
void parpadear(int pin, int veces) {
    for (int i = 0; i < veces; i++) {
        digitalWrite(pin, HIGH);
        delay(200);
        digitalWrite(pin, LOW);
        delay(200);
    }
}
```

13. Librerías comunes

```
#include <Servo.h>           // Control de servomotores
#include <Wire.h>            // Comunicación I2C
#include <SPI.h>             // Comunicación SPI
#include <Stepper.h>         // Motores paso a paso (básico)
#include <AccelStepper.h>    // Motores paso a paso (avanzado, no bloqueante)
```

Ejemplo — Servo

```
#include <Servo.h>
Servo miServo;

void setup() {
    miServo.attach(9);    // Pin de señal del servo
}

void loop() {
    miServo.write(90);    // Ángulo de 0 a 180 grados
}
```

14. Interrupciones

```
attachInterrupt(digitalPinToInterrupt(pin), funcion, MODO);
// MODO: LOW, CHANGE, RISING, FALLING
detachInterrupt(digitalPinToInterrupt(pin));
```

Una interrupción ejecuta una función automáticamente cuando ocurre un evento en un pin (ej. un botón), sin tener que estar revisando en el loop(). En Arduino Uno los pines con interrupción son el 2 y el 3.

15. Control de motores

Motor paso a paso (stepper) básico

```
#include <Stepper.h>
const int pasosPorVuelta = 200;
Stepper motor(pasosPorVuelta, 8, 9, 10, 11);

void setup() {
    motor.setSpeed(60); // RPM
}
```

```
void loop() {
  motor.step(pasosPorVuelta); // Gira una vuelta completa
}
```

16. Sensores comunes

```
// Botón / pulsador
pinMode(2, INPUT_PULLUP);
int estado = digitalRead(2); // LOW = presionado (con pull-up)

// Potenciómetro
int valor = analogRead(A0); // 0-1023

// Sensor ultrasónico HC-SR04
const int trig = 6, echo = 7;
pinMode(trig, OUTPUT);
pinMode(echo, INPUT);

digitalWrite(trig, LOW);
delayMicroseconds(2);
digitalWrite(trig, HIGH);
delayMicroseconds(10);
digitalWrite(trig, LOW);

long duracion = pulseIn(echo, HIGH);
float distanciaCm = duracion * 0.034 / 2;
```

17. Buenas prácticas y depuración

- Usa `Serial.println()` para revisar valores mientras el programa corre.
- Nombra tus variables y pines con constantes (`const int`) en vez de números sueltos ("números mágicos").
- Evita `delay()` largos si necesitas que el programa reaccione a sensores o botones al mismo tiempo; usa `millis()`.
- Verifica siempre las conexiones de GND (tierra) entre Arduino y componentes externos (drivers de motor, fuentes externas, etc.).
- Si un motor no se mueve, revisa: alimentación externa suficiente, conexiones del driver, y que el pin PWM esté bien elegido (marcado con ~).
- Comenta tu código explicando el porqué, no solo el qué.

18. Tabla resumen rápida

Función	Uso
<code>pinMode()</code>	Configurar pin como entrada/salida
<code>digitalWrite()</code> / <code>digitalRead()</code>	Escribir/leer HIGH o LOW
<code>analogWrite()</code> / <code>analogRead()</code>	PWM / lectura analógica
<code>delay()</code> / <code>millis()</code>	Control de tiempo
<code>Serial.begin()</code> / <code>print()</code> / <code>println()</code>	Comunicación con el PC
<code>map()</code> / <code>constrain()</code>	Reescalar y limitar valores
<code>attachInterrupt()</code>	Reacción inmediata a eventos
<code>Servo.attach()</code> / <code>write()</code>	Control de servomotores